

Product Roadmap

Local AI Software Team

1. Roadmap Overview

Product Name

Local AI Software Team

Product Direction

Local AI Software Team is a local-first AI software delivery tool that helps developers turn rough requirements into structured software delivery outputs.

The product starts simple:

``text id="37dva1" rough requirement → clarified requirement → technical design → task breakdown → test matrix → final report

Then gradually evolves into:

```
``text id="wyzfvg"
rough requirement
→ docs
→ implementation prompt
→ AI CLI code generation
→ tests
→ review
→ traceability
→ Jira/GitHub/Slack updates
```

The roadmap follows one key principle:

Build trust before autonomy.

The product should not start by promising fully autonomous software delivery. It should first become excellent at requirement clarification, documentation, task breakdown, and test planning. After that, it can add coding agent orchestration, test execution, review, traceability, and external integrations.

2. Roadmap Strategy

2.1 Start with a narrow wedge

The first strong niche should be:

Vietnamese/English rough requirements to structured delivery docs for Java/Spring Boot developers.

Reason:

- Requirements are often vague.
- Developers spend time clarifying business rules.
- Java/Spring Boot backend work needs API design, DB design, task planning, and testing.
- This niche avoids directly competing with broad AI coding agents.
- It matches the creator's backend experience.

2.2 Build in layers

The product should be built in layers:

Layer 1: Local project foundation Layer 2: Requirement-to-docs workflow Layer 3: Task and test planning Layer 4: Assisted coding prompt package Layer 5: AI CLI execution Layer 6: Test and review loop Layer 7: Traceability Layer 8: External integrations Layer 9: Team and enterprise features

2.3 Avoid big-bang automation

Do not build this first:

Input rough requirement → AI automatically completes full project

Build this first:

Input rough requirement → AI creates excellent delivery package

Then add controlled automation.

2.4 Roadmap stages

The roadmap is divided into:

1. Pre-MVP Foundation
2. MVP 1: Docs-only Workflow
3. MVP 2: Assisted Coding Workflow
4. MVP 3: Semi-autonomous Coding
5. MVP 4: Test, Review, and Traceability
6. Beta 1: Local Web UI Maturity
7. Beta 2: GitHub Integration
8. Beta 3: Jira and Slack Integration
9. V1.0: Stable Local AI Software Team
10. V1.x: Team Workflow
11. V2.0: Enterprise and Advanced Agent Orchestration

3. Phase 0: Pre-MVP Foundation

3.1 Goal

Build the minimum foundation needed for local workflow execution.

The goal is not to build AI automation yet.

The goal is to make the tool installable, runnable, configurable, and able to create local project artifacts.

3.2 Target User

Creator only.

3.3 Core Features

CLI foundation

Commands:

```
```bash id="48v9fl"
aiteam init
aiteam ui
aiteam doctor
aiteam config
```

## Local project initialization

Create:

```
```text id="x7hwvp" .ai-team/ config.json database.sqlite prompts/ runs/
```

Local web UI foundation

Pages:

```
```text id="djs4pg"
Dashboard
Settings
Prompt Templates
Runs
```

## Config management

Allow user to configure:

```text id="b5d2rc" project name project type default output language AI CLI command paths test/build commands safety rules

```
### Doctor check

Check:

```text id="t6a9lj"
Git
Node.js
project initialization
config validity
AI CLI availability
test command availability
```

### 3.4 Output

The product can initialize a project and show a local dashboard.

### 3.5 Success Criteria

Phase 0 is complete when:

1. User can install CLI.
2. User can run `aiteam init`.
3. `.ai-team` folder is created.
4. User can run `aiteam ui`.
5. Local web UI opens.
6. User can edit settings.
7. User can run `aiteam doctor`.
8. The system can detect basic project type.

### 3.6 Out of Scope

```text id="d667da" AI document generation code generation AI CLI execution Jira/Slack/GitHub integration traceability test runner review engine

```
---

# 4. Phase 1: MVP 1 – Docs-only Workflow

## 4.1 Goal

Turn rough requirements into structured software delivery documents.

This is the first real MVP.

The product should already be useful even without code generation.
```

4.2 Target User

- solo developer
- backend developer
- freelancer
- Vietnamese Java/Spring Boot developer

4.3 Main Promise

```text id="ibyflj"

Give the product a rough requirement.

It will generate clear requirement docs, acceptance criteria, design, tasks, test matrix, and final report.

## 4.4 Core Workflow

```text id="ym5zzg" Raw requirement → BA Agent → PO Agent → Architect Agent → PM Agent → QA Agent → Reporter Agent → Final docs

4.5 Required CLI Commands

```bash id="lv0lw9"

aiteam new

aiteam run --mode docs-only

aiteam spec

aiteam scope

aiteam plan

aiteam tasks

aiteam tests

aiteam report

aiteam list

aiteam show

## 4.6 Required Local Web UI

Pages:

```text id="rpq1fw" Dashboard New Requirement Runs Run Detail Settings Prompt Templates

Run Detail tabs:

```text id="pkv85l"

Overview

Input

Requirement

Scope

Design  
Tasks  
Tests  
Report  
Logs

## 4.7 Generated Artifacts

```text id="tvqfwb" input.md

requirement/ clarified-requirement.md business-rules.md acceptance-criteria.md open-questions.md assumptions.md

scope/ scope-definition.md out-of-scope.md success-criteria.md

design/ technical-design.md api-design.md db-design.md service-flow.md risk-analysis.md

tasks/ task-breakdown.md task-breakdown.json jira-ready-tasks.md

tests/ test-matrix.md manual-test-checklist.md

report/ final-report.md

4.8 Required Agent Roles

```text id="xj5tqb"

BA Agent

Product Owner Agent

Architect Agent

Project Manager Agent

QA Agent

Reporter Agent

## 4.9 Product Requirements

The system must:

1. Accept rough requirement.
2. Detect input language.
3. Generate clarified requirement.
4. Generate business rules.
5. Generate acceptance criteria.
6. Generate open questions and assumptions.
7. Generate MVP scope and out-of-scope list.
8. Generate technical design.
9. Generate API/DB design if relevant.
10. Generate task breakdown.

11. Generate test matrix.
12. Generate final report.
13. Save all outputs locally.
14. Show outputs in local web UI.
15. Allow copy/export markdown.

## 4.10 Success Criteria

MVP 1 is successful when:

1. A user can enter a vague requirement and get useful docs.
2. The docs are better than a generic one-shot AI chat response.
3. The task breakdown is usable for Jira/manual planning.
4. The test matrix covers major acceptance criteria.
5. User can view and export all artifacts locally.
6. No code is modified.

## 4.11 Example Demo

Input:

```
```text id="8gpp82" Thêm API export invoice CSV theo hotelId, date range, status.
```

Output:

```
```text id="wcj2v4"
clarified requirement
acceptance criteria
API design
task breakdown
test matrix
final report
```

## 4.12 Out of Scope

```
```text id="kgcw8d" AI CLI code execution diff viewer test execution review engine GitHub PR creation Jira
API integration Slack integration multi-agent parallel execution
```

```
# 5. Phase 2: MVP 2 – Assisted Coding Workflow
```

```
## 5.1 Goal
```

Generate high-quality implementation prompts for existing AI coding agents.

The product still does not run coding agents automatically.

5.2 Main Promise

```text id="s3qbh5"

The product creates a complete implementation prompt package that the user can copy into Claude Code, Codex CLI, Gemini CLI, Aider, Cursor, or another coding tool.

## 5.3 Target User

- developers already using AI coding tools
- users who want safety and control
- users not ready for auto-code

## 5.4 Core Workflow

```text id="yqoqct" Raw requirement → Docs → Technical plan → Test matrix → Implementation prompt  
→ Coding checklist → User manually runs AI coding agent

5.5 New Features

Implementation Prompt Generator

Generates:

```text id="nsl5ft"

implementation-prompt.md  
coding-checklist.md  
expected-files.md  
manual-test-checklist.md

## Copy Prompt UX

User can copy:

```text id="u9kt70" Full implementation prompt Short implementation prompt Agent-specific prompt

Agent-specific prompt style

Support prompt format for:

```text id="4rw6aj"

Claude Code  
Codex CLI  
Gemini CLI  
Aider  
Generic agent



## 5.6 New CLI Commands

```bash id="a4p72r" aiteam code --prompt-only aiteam prompts

```
## 5.7 New UI Features
```

In Run Detail:

```
```text id="r0ohl5"  
Implementation tab
Implementation prompt viewer
Copy prompt button
Agent-specific prompt selection
Coding checklist viewer
```

## 5.8 Success Criteria

Phase 2 is complete when:

1. User can generate implementation prompt from approved docs.
2. User can choose target coding agent style.
3. User can copy the prompt easily.
4. Prompt includes requirement, acceptance criteria, design, constraints, tests, and safety rules.
5. User can manually run the prompt in external AI coding tool.

## 5.9 Out of Scope

```text id="w38v9u" Running AI CLI automatically collecting diff automatically running tests  
automatically reviewing generated code automatically

```
---
```

```
# 6. Phase 3: MVP 3 – Semi-autonomous AI CLI Execution
```

```
## 6.1 Goal
```

Allow the product to run one selected AI coding CLI after user approval.

This is the first coding automation phase.

```
## 6.2 Main Promise
```

```
```text id="paan8x"
```

Approve the plan, choose an AI coding CLI, and the product will run it locally, collect changed files, and generate a diff.

## 6.3 Supported AI CLIs

Start with one adapter first.

Recommended first adapter:

```
```text id="x11scf" Codex CLI or Claude Code
```

Then add:

```
```text id="pj3gzm"
Gemini CLI
Aider
```

## 6.4 Core Workflow

```
```text id="021sc0" Docs → Plan approval → Implementation prompt → User approval → Run selected AI
CLI → Collect changed files → Generate diff patch → Save logs
```

```
## 6.5 Required Features
```

```
### AI CLI adapter layer
```

Supports:

```
```text id="6g84co"
isAvailable()
runTask()
collectOutput()
collectChangedFiles()
handleTimeout()
```

### Approval before code generation

User must approve:

```
```text id="m3iw33" selected agent working directory prompt affected areas timeout safety rules
```

```
### Git snapshot
```

Before AI runs:

```
```text id="24mfy5"
save current git status
```

```
save current diff
record checkpoint
```

## Diff collection

After AI runs:

```
```text id="nj4f6p" changed-files.json diff.patch agent-output.log implementation-notes.md
```

```
## 6.6 Required CLI Commands

```bash id="pu4zun"
aiteam code --run <runId> --agent codex
aiteam approve <runId> --gate code
aiteam rollback <runId> --dry-run
```

## 6.7 Required UI Features

```
```text id="vcskwk" Code generation approval modal Implementation prompt preview Agent availability
status Agent log viewer Changed files viewer Diff viewer basic Rollback preview
```

```
## 6.8 Safety Requirements

Must protect:

```text id="xuv74s"
.env
.env.*
*.pem
*.key
credentials.json
application-prod.yml
application-production.yml
```

Must block:

```
```text id="gmfppt" sudo rm -rf / chmod 777 curl | sh wget | sh mkfs dd if=
```

```
## 6.9 Success Criteria

Phase 3 is complete when:

1. User can approve code generation.
2. Product can run one AI coding CLI.
3. Product saves logs.
4. Product collects changed files.
```

5. Product generates diff patch.
6. Product detects protected file modifications.
7. Product can show output in UI.
8. Product does not auto-commit or auto-push.

6.10 Out of Scope

```
```text id="l3jsyh"
multi-agent code execution
automatic PR creation
automatic test fixing loop
Jira/Slack updates
production deployment
```

---

## 7. Phase 4: MVP 4 — Test, Review, and Traceability

### 7.1 Goal

Make AI-generated work more trustworthy.

The product should not only generate code. It should test, review, and map work back to requirements.

### 7.2 Main Promise

```text id="zdi31y" After AI generates code, the product runs tests, reviews the diff, checks coverage, and creates a traceability matrix.

7.3 Core Workflow

```
```text id="zoq6c3"
AI-generated code
→ Run build/tests
→ Summarize test result
→ Review diff
→ Security review
→ Requirement coverage
→ Traceability matrix
→ Final report
```

## 7.4 Required Features

### Test Runner

Runs configured commands:

```
```text id="4dc1ao" build unit test integration test lint type check custom command
```

```
### Test Result Summary
```

Generates:

```
```text id="l09yq5"
test-result.md
failed-tests.md
test.log
```

### Review Assistant

Checks:

```
```text id="3i8s14" requirement coverage code quality test coverage security risks unrelated changes
protected file changes missing edge cases
```

```
### Traceability Engine
```

Maps:

```
```text id="frsmag"
Requirement
→ Acceptance Criteria
→ Task
→ Code Files
→ Test Cases
→ Test Result
```

## 7.5 Required CLI Commands

```
```bash id="4oglop" aiteam test aiteam review aiteam trace aiteam report
```

```
## 7.6 Required UI Features
```

```
```text id="l43bks"
Test Result tab
Review tab
Traceability tab
```

Failed tests viewer  
Requirement coverage table  
Security review summary

## 7.7 Success Criteria

Phase 4 is complete when:

1. User can run configured tests.
2. Product summarizes test result.
3. Product generates review report.
4. Product generates security review.
5. Product creates traceability matrix.
6. Product can generate final delivery report after code/test/review.
7. User can tell which requirements are covered, partial, or not covered.

## 7.8 Out of Scope

``text id="5azsx0" automatic fix loop beyond one manual rerun PR creation CI integration Jira/Slack integration team collaboration

```

8. Phase 5: Beta 1 – Local Web UI Maturity

8.1 Goal

Improve the local web UI into a polished control center.

8.2 Main Promise

``text id="ce9bax"
Users can manage the whole local AI software team workflow visually from the browser.
```

## 8.3 Required Improvements

### Dashboard

Add:

``text id="36ohiq" project status AI CLI status latest runs quick actions local mode indicator

### Run Detail

Improve:

```
``text id="7ucaff"
workflow timeline
artifact tabs
markdown viewer
logs viewer
diff viewer
test result viewer
traceability table
approval screens
```

## Settings

Improve:

```
``text id="586t7s" project settings AI CLI settings agent mapping workflow settings test commands
safety settings documentation settings storage settings
```

```
Prompt Template Editor

Add:

``text id="3ontfr"
markdown editor
variable validation
preview
reset to default
duplicate template
```

## 8.4 UX Requirements

The UI should make these obvious:

```
``text id="6ekxgj" What has been generated? What is running now? What failed? What needs approval?
What is the next recommended action?
```

### ## 8.5 Success Criteria

Beta 1 is complete when:

1. User can run docs-only workflow from UI.
2. User can review all generated artifacts.
3. User can configure settings without editing JSON.
4. User can edit prompt templates.
5. User can view logs.
6. User can view diff and test results if available.
7. UI feels stable enough for private beta users.

---

## # 9. Phase 6: Beta 2 – GitHub Integration

### ## 9.1 Goal

Support GitHub workflow after local code generation.

### ## 9.2 Main Promise

```text id="ofd9yx"

After local implementation and review, the product can help create a GitHub PR summary and optionally create a PR with user approval.

9.3 Recommended MVP Integration Method

Use GitHub CLI first:

```text id="1tz2m9" gh auth login

Then call:

```text id="l6xlp4"

gh pr create

gh pr view

gh run list

9.4 Features

GitHub status check

Show:

```text id="vdd89f" GitHub CLI installed authentication status current repo remote origin current branch

### ### PR summary generator

Generate:

```text id="uyljti"

PR title

PR description

changed files summary

test result summary

traceability summary

risk notes

Optional PR creation

With approval:

```text id="zbyxt6" create branch create commit push branch create PR

### ## 9.5 Required CLI Commands

```
```bash id="tdbujn"
aiteam github status
aiteam github pr create --run <runId>
aiteam github actions --run <runId>
```

9.6 UI Features

```text id="yajyqd" GitHub integration settings PR summary preview Create PR approval modal GitHub Actions result viewer

### ## 9.7 Success Criteria

Phase 6 is complete when:

1. Product can detect GitHub CLI status.
2. Product can generate PR summary.
3. Product can create PR after approval.
4. Product can read basic CI status.
5. Product links final report to PR summary.

### ## 9.8 Out of Scope

```
```text id="mcqyao"
auto merge
auto deployment
complex branch strategy
multi-repo PR
enterprise GitHub admin
```

10. Phase 7: Beta 3 — Jira and Slack Integration

10.1 Goal

Make the product feel more like a real software team connected to team tools.

10.2 Main Promise

```text id="dm7wj" The product can generate Jira-ready tasks, optionally create Jira issues, and post workflow updates to Slack after user approval.

```

10.3 Jira Roadmap

Step 1: Jira-ready Markdown

Generate copyable Jira-ready task output.

No API required.

Output:

```text id="r694c8"
Epic
Stories
Subtasks
Acceptance criteria
Definition of done
```

Step 2: Jira API test connection

User configures:

```text id="pp0yjt" Jira site URL email API token project key default issue type

```
Step 3: Create Jira issues

With approval:

```text id="j574rl"
create epic
create story
create subtask
link parent-child relation
add final report comment
```

Step 4: Jira workflow update

Later:

```text id="7mu9ab" update issue status comment progress link GitHub PR attach docs

---

## ## 10.4 Slack Roadmap

### ### Step 1: Post-only Slack

MVP Slack should only post messages.

Supported messages:

```
```text id="09u2bn"
workflow started
docs generated
code generated
tests failed
PR ready
final report ready
```

Step 2: Final report summary

Post final result to configured channel.

Step 3: Approval messages

Later:

```
```text id="mtwj06" send approval request message
```

### ### Step 4: Slack event receiving

Not recommended for local-only MVP because it requires public callback URL.

Future options:

```
```text id="01c49e"
ngrok
Cloudflare Tunnel
Tailscale Funnel
future cloud backend
```

10.5 Required CLI Commands

```
```bash id="d1vt55" aiteam jira export --run <runId> aiteam jira test aiteam jira create --run <runId>
```

```
aiteam slack test aiteam slack post --run <runId>
```

## ## 10.6 UI Features

```text id="jrszog"

Jira settings

Jira-ready task preview

Create Jira tasks approval modal

Slack settings

Slack message preview

Post to Slack approval modal

10.7 Success Criteria

Phase 7 is complete when:

1. Product can generate Jira-ready tasks.
2. Product can test Jira connection.
3. Product can create Jira issues after approval.
4. Product can test Slack connection.
5. Product can post final report to Slack after approval.
6. No external update happens silently.

10.8 Out of Scope

```text id="2nfcti" Slack bot commands Slack event receiving full Jira workflow automation enterprise approval routing advanced permission mapping

---

## # 11. Phase 8: V1.0 – Stable Local AI Software Team

### ## 11.1 Goal

Release a stable version for individual developers and small teams.

### ## 11.2 Main Promise

```text id="y6e235"

A reliable local AI software team for turning rough requirements into docs, tasks, code prompts, optional code changes, tests, reviews, traceability, and final reports.

11.3 V1.0 Feature Set

Core

``text id="vk70ny" local CLI local web UI local project config run history prompt templates docs-only workflow assisted workflow semi-auto workflow test runner review engine traceability matrix final report

AI CLIs

Support at least:

``text id="iykdgi"

Claude Code

Codex CLI

Gemini CLI

Aider

Project Templates

Support:

``text id="a6kvoh" Java / Spring Boot Node.js / NestJS React / Vite Generic

Safety

Include:

``text id="p9mtu8"

protected files

blocked commands

approval gates

secret masking

rollback preview

Git snapshot

local logs

Export

Support:

``text id="gu5tk3" Markdown export ZIP export JSON export

Optional Integrations

Stable basic support:

```
```text id="uo4z8i"
GitHub PR summary / PR creation
Jira-ready export / basic issue creation
Slack post final report
```

## 11.4 V1.0 Acceptance Criteria

V1.0 is ready when:

1. User can install the product easily.
2. User can initialize a local project.
3. User can run docs-only workflow reliably.
4. User can run assisted workflow reliably.
5. User can run semi-auto workflow with at least one AI CLI.
6. User can run tests and view results.
7. User can view review report.
8. User can view traceability matrix.
9. User can export final delivery package.
10. Safety rules work by default.
11. Generated outputs are consistently useful.
12. Product works without backend cloud.
13. Product works without desktop app.

---

## 12. Phase 9: V1.x — Team Workflow

### 12.1 Goal

Support small teams while still keeping local-first philosophy.

### 12.2 Main Promise

```text id="auedy4" Small teams can standardize requirement-to-delivery workflows using shared templates, policy files, and optional external integrations.

```
## 12.3 Features
```

```
### Shared templates
```

Support export/import for:

```
```text id="43sghl"
prompt templates
workflow presets
safety rules
```

```
documentation templates
project templates
```

## Team policy file

Example:

```
```text id="ji2gq5" .ai-team/policy.json
```

Policy can define:

```
```text id="86dxyc"
approval rules
protected files
allowed commands
required docs
required test commands
required review steps
```

## Audit report

Generate:

```
```text id="8qmnla" who approved what what was generated what commands ran what files changed
what external updates happened
```

```
### Multi-project support

UI supports multiple local projects.

### Team output standardization

Templates for:

```text id="5zeos3"
backend feature
bug fix
refactor
API change
DB migration
security patch
```

## 12.4 Success Criteria

V1.x team workflow is successful when:

1. A small team can share templates.
2. A team can define safety policies.
3. Generated docs follow team standards.
4. Audit report is available.
5. External updates are controlled.
6. Team members can use the same workflow locally.

## 12.5 Out of Scope

central SaaS workspace real-time collaboration SSO central admin enterprise user management

---

# 13. Phase 10: V2.0 – Advanced Agent Orchestration

## 13.1 Goal

Move from role-based prompt workflow to stronger AI agent orchestration.

## 13.2 Main Promise

``text id="cyb9x2"

The product can coordinate multiple AI agents with stronger planning, context handoff, fix loops, and evaluation.

## 13.3 Features

### Agent handoff

Each agent produces structured output for the next agent.

Example:

BA Agent output → Architect Agent input Architect Agent output → Developer Agent input QA Agent output → Reviewer Agent input

### Multi-agent evaluation

Add reviewer agents for:

``text id="35f59g"

requirement quality

design quality



```
task quality
test quality
code quality
security
```

## Fix loop

Controlled loop:

``text id="y38dj0" test failure → QA summarizes issue → Developer fixes → tests run again → reviewer checks → stop on pass or max iterations

```
Context management
```

Improve:

```
``text id="jduokm"
repo context selection
file relevance detection
prompt size control
artifact summarization
memory per run
```

## Agent scoring

Score outputs:

``text id="xrvul7" clarity completeness testability risk confidence

```
Workflow presets
```

Presets:

```
``text id="74f9dh"
Backend API Feature
Bug Fix
Refactor
DB Migration
Security Patch
Frontend Feature
Full-stack Feature
```

## 13.4 Success Criteria

V2.0 is successful when:

1. Multi-agent workflow produces better outputs than single-agent workflow.
2. Fix loop can resolve common test failures.
3. Context passed to agents is concise and relevant.
4. Review quality improves.
5. User trust increases because each stage is explainable.

---

## 14. Phase 11: V2.x — Enterprise / Self-hosted Direction

### 14.1 Goal

Support larger organizations that need central policy, compliance, and stronger security.

### 14.2 Possible Direction

At this stage, the product may need one of these:

``text id="tpud5o" self-hosted backend local network server enterprise control plane private model integration

This is not needed for MVP or V1.

#### ## 14.3 Enterprise Features

Possible features:

``text id="kg7t94"

SSO

RBAC

central policy management

central audit logs

team license management

internal prompt registry

private model support

on-prem Git integration

Jira/Confluence enterprise integration

security approval workflow

### 14.4 Enterprise Constraints

Enterprise mode must address:

```text id="t8bmbj" source code privacy secret handling auditability approval policies tool access control network restrictions model provider restrictions

14.5 Success Criteria

Enterprise direction is worth pursuing only if:

1. Individual/local product has real users.
2. Small teams request shared control.
3. Companies need centralized policy.
4. There is willingness to pay.
5. Local-first version has strong trust.

15. Milestone Timeline

15.1 Month 1: Foundation + Docs-only Prototype

Focus:

```text id="2kyqhd"  
CLI init  
local web UI  
settings  
prompt templates  
new requirement  
docs-only workflow  
markdown output

Deliverable:

```text id="l4cat4" User can input rough requirement and get requirement/design/tasks/tests/report.

15.2 Month 2: MVP Polish + Java/Spring Boot Template

Focus:

```text id="t92c4p"  
better prompt templates  
Java/Spring Boot project template  
run history  
docs viewer  
task breakdown quality  
test matrix quality  
export

Deliverable:

```text id="xhq3y" Private MVP for Java/Spring Boot developers.

15.3 Month 3: Assisted Coding

Focus:

```text id="20lpge"  
implementation prompt generator  
agent-specific prompts  
coding checklist  
prompt copy UX  
manual review flow

Deliverable:

```text id="owivm7" User can generate high-quality prompts for Claude Code/Codex/Gemini/Aider.

15.4 Month 4: AI CLI Execution

Focus:

```text id="swuk99"  
one AI CLI adapter  
approval gate  
agent logs  
changed files  
diff patch  
protected file detection

Deliverable:

```text id="uz5pz4" User can approve and run one AI coding CLI locally.

15.5 Month 5: Test + Review + Traceability

Focus:

```text id="koi0zp"  
test runner  
review report  
security review  
traceability matrix  
final report after coding

Deliverable:

```text id="u6qk5w" User can verify AI-generated changes.

15.6 Month 6: Beta Release

Focus:

```text id="rl8n9q"

UI polish

bug fixing

export

GitHub basic support

Jira-ready tasks

Slack post-only

documentation

landing page

Deliverable:

```text id="exzhky" Public beta or private paid beta.

16. Feature Priority Matrix

16.1 Must-have for MVP

```text id="oqgzvm"

CLI init

local web UI

settings

new requirement

docs-only workflow

requirement clarification

acceptance criteria

technical design

task breakdown

test matrix

final report

run history

markdown export

prompt templates

## 16.2 Should-have for MVP+

```text id="5uu865" implementation prompt agent-specific prompt repository analyzer Java/Spring Boot  
template traceability before code CLI doctor logs viewer prompt template editor

```
## 16.3 Could-have
```

```
```text id="9ccov6"
```

```
AI CLI execution
```

```
diff viewer
```

```
test runner
```

```
review report
```

```
security review
```

```
GitHub PR summary
```

```
Jira-ready export
```

```
Slack final report post
```

## 16.4 Later

```text id="eehhsx" multi-agent orchestration fix loop GitHub PR creation Jira issue creation Slack event receiving Confluence/Notion publishing team policy audit report enterprise features

```
---
```

```
# 17. Risk-based Roadmap Control
```

```
## 17.1 Biggest Product Risk
```

```
Users may ask:
```

```
```text id="b2y9bk"
```

```
Why not just use Claude Code or ChatGPT directly?
```

Mitigation:

```text id="exx3kz" Make workflow outputs structured, reusable, traceable, and better than one-shot prompts.

```
## 17.2 Biggest Technical Risk
```

```
AI CLI execution may be unstable and unsafe.
```

```
Mitigation:
```

```
```text id="xfxzlr"
```

```
Delay auto-code until docs-only and assisted workflows are strong.
```

```
Add approval gates, protected files, diff review, and rollback.
```

## 17.3 Biggest Scope Risk

The product can become too broad.

Mitigation:

```text id="tld74u" Do not start with Jira, Slack, GitHub, CI, deployment, and full autonomy. Start with requirement-to-docs.

17.4 Biggest GTM Risk

Market may see it as "another AI coding tool."

Mitigation:

```text id="df9cp5"

Position as requirement-to-delivery workflow, not coding agent.

---

## 18. Recommended Build Order

The recommended build order is:

```text id="0g4v7h" 1. CLI init 2. Local web UI settings 3. Prompt templates 4. New requirement page 5. Docs-only workflow 6. Run history 7. Docs viewer 8. Export 9. Java/Spring Boot template 10. Implementation prompt generator 11. Repository analyzer 12. AI CLI adapter 13. Diff viewer 14. Test runner 15. Review engine 16. Traceability 17. GitHub 18. Jira 19. Slack 20. Team policies

Do not build integrations before the core workflow is useful.

19. Demo Roadmap

The product should have demo scenarios for every milestone.

Demo 1: Invoice CSV Export

Input:

```text id="981dzo"

Thêm API export invoice CSV theo hotelId, date range, status.

Used to test:

```text id="yn805a" requirement clarification API design task breakdown test matrix implementation prompt

Demo 2: Reset Password by OTP

Input:

```
```text id="aay5rp"
Thêm chức năng reset password bằng email OTP.
```

Used to test:

```text id="81lps1" security considerations validation rules test scenarios code generation later

Demo 3: Booking Cancellation Approval

Input:

```
```text id="q0lxa8"
Tạo approval flow cho booking cancellation. Nếu refund > 50 triệu thì cần
director duyệt.
```

Used to test:

```text id="mnckaa" business rules approval workflow edge cases traceability

Demo 4: Expense Approval System

Input:

```
```text id="f0hg7y"
Build an expense approval system. Employee submits request, manager approves,
finance pays, admin views report.
```

Used to test:

```text id="3fv06c" larger feature planning task breakdown scope control

20. V1 Launch Checklist

Before V1 launch, the product should have:

Product

```
```text id="mpe9u6"
clear positioning
landing page
demo video
sample outputs
```



documentation  
installation guide

## Core Quality

``text id="n2zltk" stable CLI stable local UI good docs output good task output good test matrix output  
safe defaults local storage reliable

```
Developer Experience

``text id="y98nml"
npm install works
aiteam init works
aiteam ui works
aiteam doctor works
error messages are clear
logs are accessible
```

## Safety

``text id="s5ckn8" protected file rules blocked command rules approval gates secret masking rollback  
preview

```
Support

``text id="10tahm"
README
quickstart
troubleshooting
example project
demo scenarios
known limitations
```

---

## 21. Final Roadmap Summary

The roadmap should move in this order:

``text id="kykg6k" 1. Local foundation 2. Docs-only workflow 3. Assisted coding prompt package 4. AI  
CLI execution 5. Test/review/traceability 6. Local web UI maturity 7. GitHub 8. Jira/Slack 9. V1 stable  
release 10. Team workflow 11. Advanced multi-agent orchestration 12. Enterprise/self-hosted later

The most important rule:

> Do not start with full autonomy. Start with structured clarity.

The first product users should love this output:

```
```text id="i64m3i"
rough requirement
→ clear requirement
→ business rules
→ acceptance criteria
→ technical design
→ task breakdown
→ test matrix
→ final report
```

Only after this is strong should the product move into:

```
text id="u2xq7r"
AI CLI execution
→ code patch
→ tests
→ review
→ traceability
→ PR/Jira/Slack
```

The strongest version of this product is not "AI writes code."

The strongest version is:

A local AI software team that brings structure, safety, and traceability to AI-assisted software delivery.